

SFT and LoRA

Supervised fine-tuning, and training a fraction of the weights

Carlos Cotrini

ETH Zurich

7 October 2026

1

Section 1

What supervised fine-tuning changes

A base model continues the document

The same question

o o o

How do I see my jobs in the Slurm queue?

A base model continues the document

The same question

ooo

How do I see my jobs in the Slurm queue?

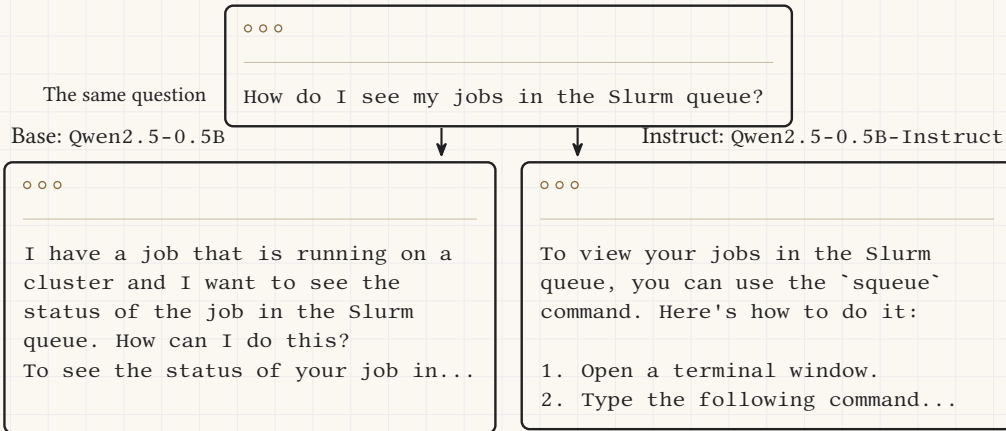
Base: Qwen2.5-0.5B



ooo

I have a job that is running on a cluster and I want to see the status of the job in the Slurm queue. How can I do this?
To see the status of your job in...

A base model continues the document

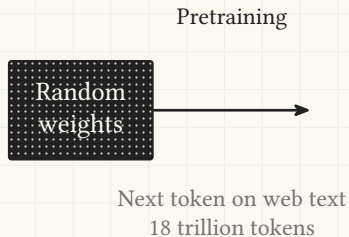


The same network, one more training stage



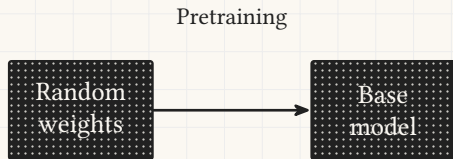
Random
weights

The same network, one more training stage



Qwen2.5 Technical Report, arXiv:2412.15115

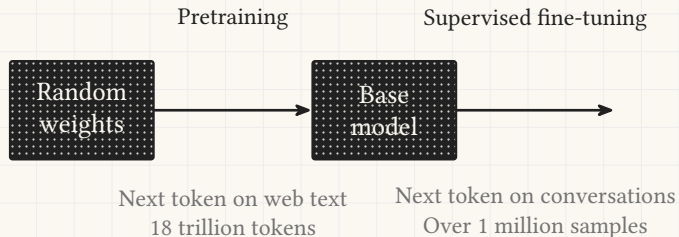
The same network, one more training stage



Next token on web text
18 trillion tokens

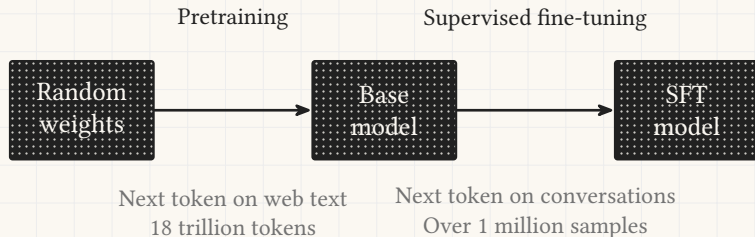
Qwen2.5 Technical Report, arXiv:2412.15115

The same network, one more training stage



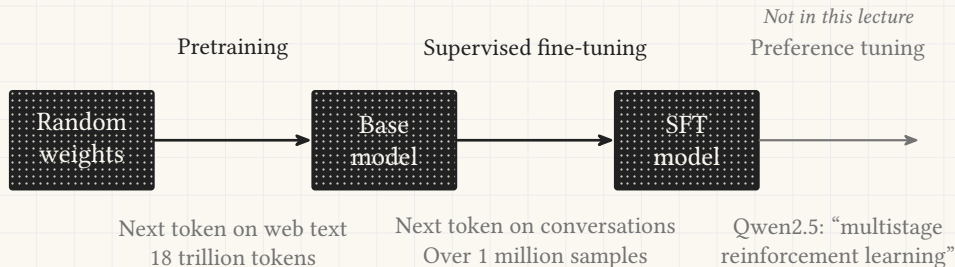
Qwen2.5 Technical Report, arXiv:2412.15115

The same network, one more training stage



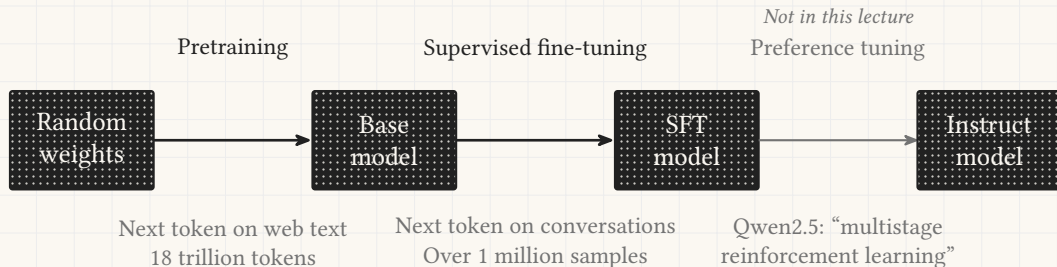
Qwen2.5 Technical Report, arXiv:2412.15115

The same network, one more training stage



Qwen2.5 Technical Report, arXiv:2412.15115

The same network, one more training stage

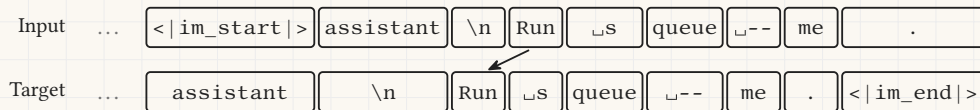


Qwen2.5 Technical Report, arXiv:2412.15115

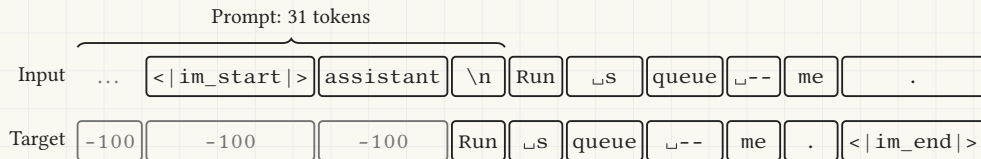
The loss counts only the answer

Input ... `<|im_start|>` `assistant` `\n` `Run` `_s` `queue` `_--` `me` `.`

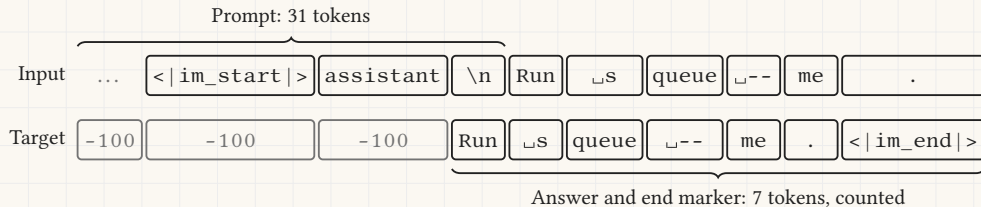
The loss counts only the answer



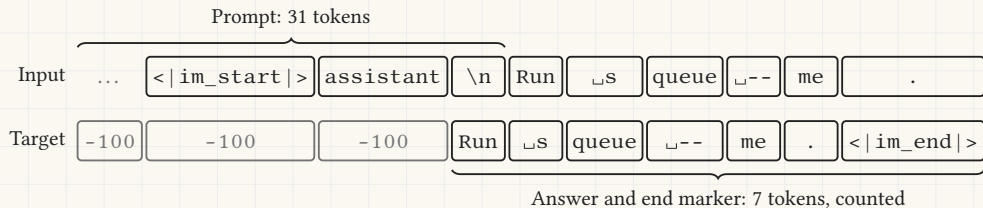
The loss counts only the answer



The loss counts only the answer

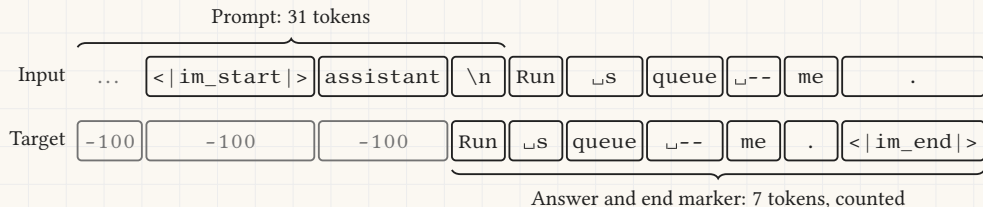


The loss counts only the answer



$$\underbrace{\left(\log \hat{y}_{31, \text{Run}} + \log \hat{y}_{32, \text{▯s}} + \cdots + \log \hat{y}_{37, <|im_end|>} \right)}_{\text{the 7 counted targets}}$$

The loss counts only the answer



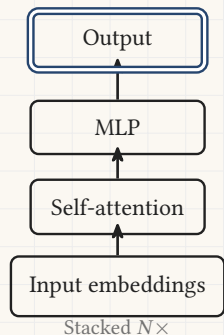
$$\bar{l} = \underbrace{-\frac{1}{7}}_{\text{mean}} \underbrace{\left(\log \hat{y}_{31, \text{Run}} + \log \hat{y}_{32, \text{▯s}} + \cdots + \log \hat{y}_{37, <|im_end|>} \right)}_{\text{the 7 counted targets}}$$

2

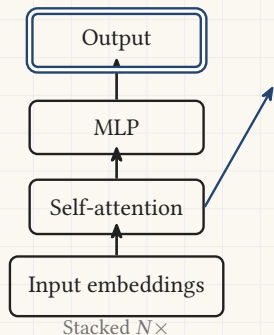
Section 2

From pretraining to adapting

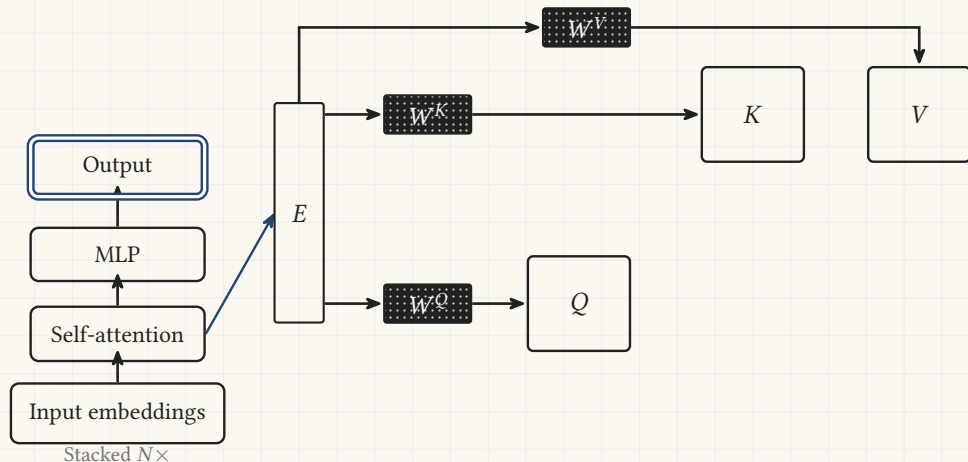
The transformer block, and self-attention



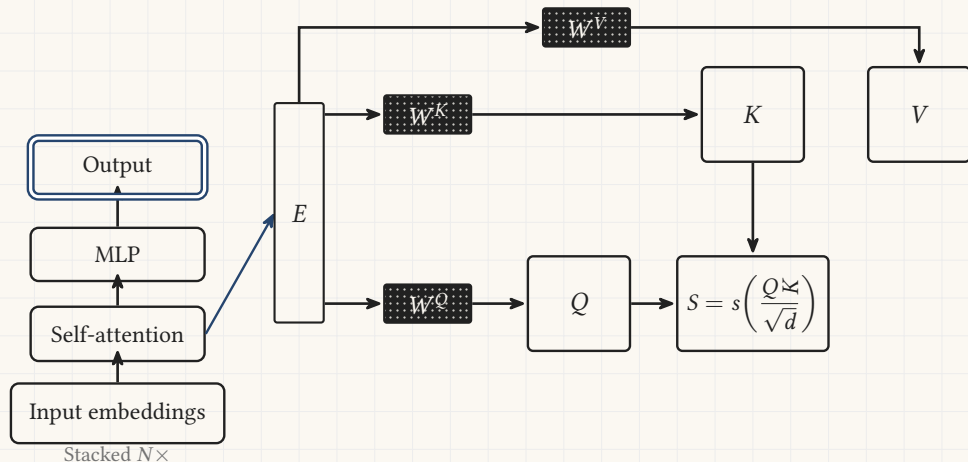
The transformer block, and self-attention



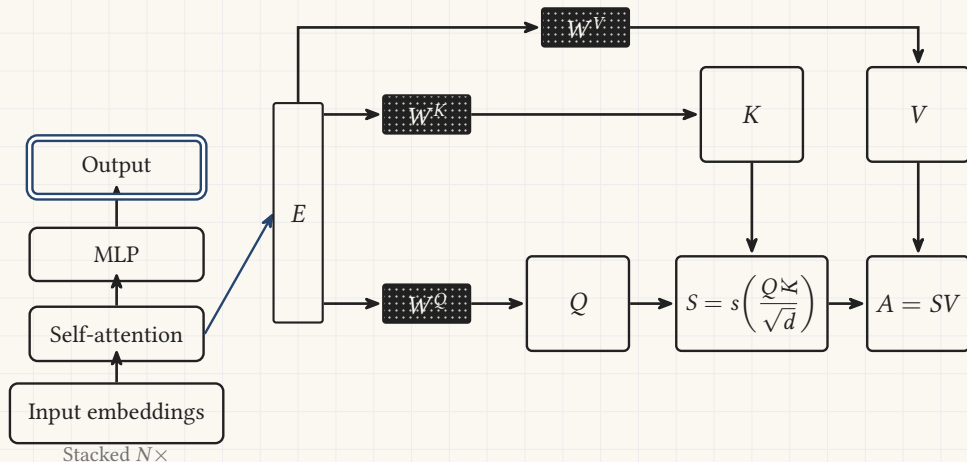
The transformer block, and self-attention



The transformer block, and self-attention



The transformer block, and self-attention



From pretraining to fine-tuning

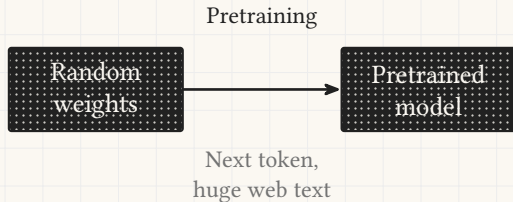


Random
weights

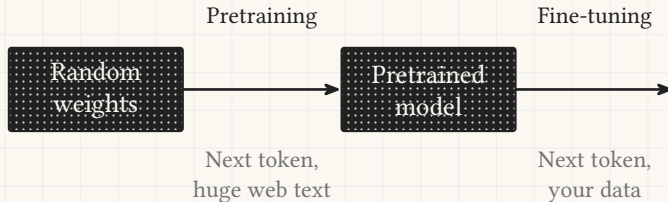
From pretraining to fine-tuning



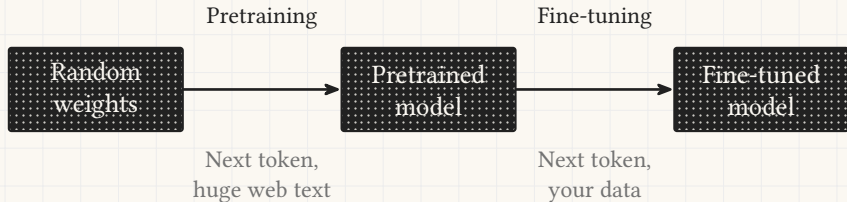
From pretraining to fine-tuning



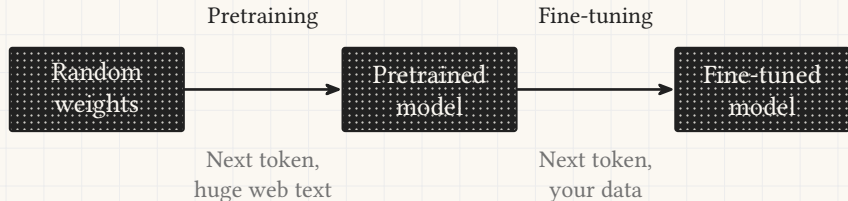
From pretraining to fine-tuning



From pretraining to fine-tuning

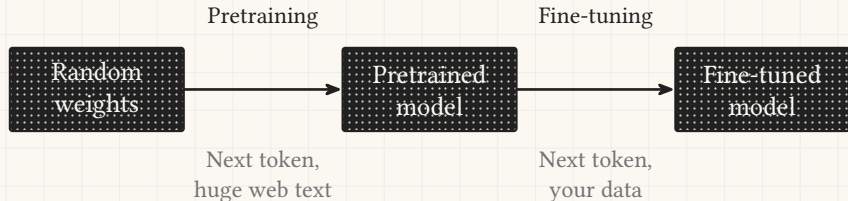


From pretraining to fine-tuning



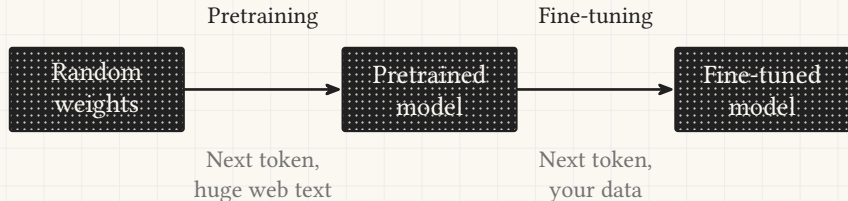
- Same recipe: gradient descent, the same kind of loss

From pretraining to fine-tuning



- Same recipe: gradient descent, the same kind of loss
- Starts from the pretrained weights, not random ones

From pretraining to fine-tuning



- Same recipe: gradient descent, the same kind of loss
- Starts from the pretrained weights, not random ones
- Trains on your data, not the whole web

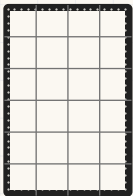
Freeze the matrix, train a small update

New data for fine-tuning

Freeze the matrix, train a small update

New data for fine-tuning

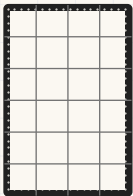
Naive: retrain every parameter



Freeze the matrix, train a small update

New data for fine-tuning

Naive: retrain every parameter

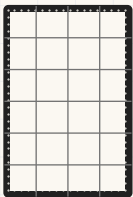


8.05 billion parameters

Freeze the matrix, train a small update

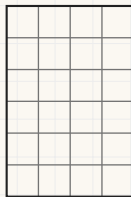
New data for fine-tuning

Naive: retrain every parameter



8.05 billion parameters

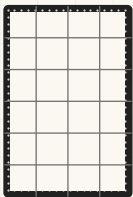
Instead: freeze W



Freeze the matrix, train a small update

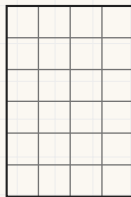
New data for fine-tuning

Naive: retrain every parameter



8.05 billion parameters

Instead: freeze W

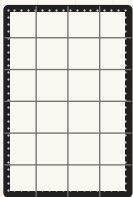


Train only this

Freeze the matrix, train a small update

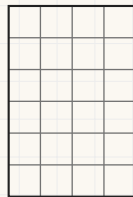
New data for fine-tuning

Naive: retrain every parameter



8.05 billion parameters

Instead: freeze W



Train only this

$\Delta W = BA$: few numbers

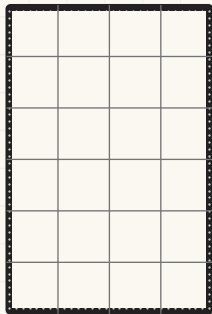
3

Section 3

One matrix, and a low-rank update

Twenty-four numbers, or ten

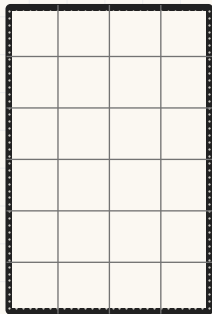
W : 6 outputs $\times$ 4 inputs



All 24 are updated

Twenty-four numbers, or ten

W : 6 outputs $\times$ 4 inputs



All 24 are updated

- Full fine-tuning updates all 24

Twenty-four numbers, or ten

W : 6 outputs $\times$ 4 inputs

Frozen: no gradient,
no moments

B : 6×1

b_1
b_2
b_3
b_4
b_5
b_6

- Full fine-tuning updates all 24

Twenty-four numbers, or ten

W : 6 outputs $\times$ 4 inputs

Frozen: no gradient,
no moments

A : 1×4

a_1	a_2	a_3	a_4
-------	-------	-------	-------

B : 6×1

b_1
b_2
b_3
b_4
b_5
b_6

- Full fine-tuning updates all 24

Twenty-four numbers, or ten

W : 6 outputs $\times$ 4 inputs

Frozen: no gradient,
no moments

B : 6×1

A : 1×4				
	a_1	a_2	a_3	a_4
b_1	$b_1 a_1$	$b_1 a_2$	$b_1 a_3$	$b_1 a_4$
b_2	$b_2 a_1$	$b_2 a_2$	$b_2 a_3$	$b_2 a_4$
b_3	$b_3 a_1$	$b_3 a_2$	$b_3 a_3$	$b_3 a_4$
b_4	$b_4 a_1$	$b_4 a_2$	$b_4 a_3$	$b_4 a_4$
b_5	$b_5 a_1$	$b_5 a_2$	$b_5 a_3$	$b_5 a_4$
b_6	$b_6 a_1$	$b_6 a_2$	$b_6 a_3$	$b_6 a_4$

$$\Delta W = BA$$

- Full fine-tuning updates all 24
- LoRA freezes W and learns B and A : $6 + 4 = 10$ numbers, a ratio of 2.4

Twenty-four numbers, or ten

W : 6 outputs $\times$ 4 inputs

Frozen: no gradient,
no moments

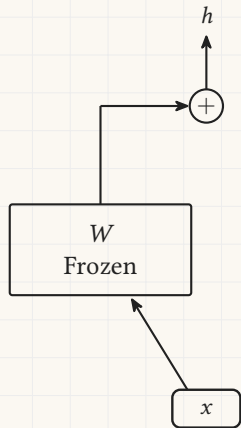
B : 6 $\times$ 1

A : 1 $\times$ 4				
	a_1	a_2	a_3	a_4
b_1	$b_1 a_1$	$b_1 a_2$	$b_1 a_3$	$b_1 a_4$
b_2	$b_2 a_1$	$b_2 a_2$	$b_2 a_3$	$b_2 a_4$
b_3	$b_3 a_1$	$b_3 a_2$	$b_3 a_3$	$b_3 a_4$
b_4	$b_4 a_1$	$b_4 a_2$	$b_4 a_3$	$b_4 a_4$
b_5	$b_5 a_1$	$b_5 a_2$	$b_5 a_3$	$b_5 a_4$
b_6	$b_6 a_1$	$b_6 a_2$	$b_6 a_3$	$b_6 a_4$

$$\Delta W = BA$$

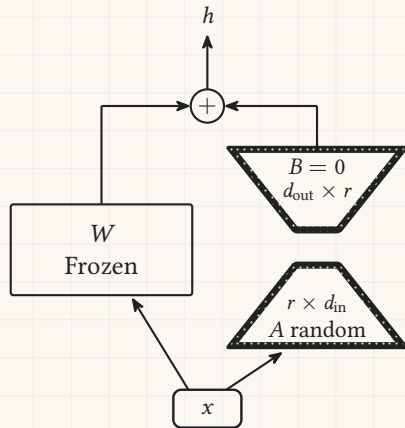
- Full fine-tuning updates all 24
- LoRA freezes W and learns B and A : $6 + 4 = 10$ numbers, a ratio of 2.4
- Every row of BA is the row A , rescaled. That is what rank one means

Both paths run, and their outputs are added



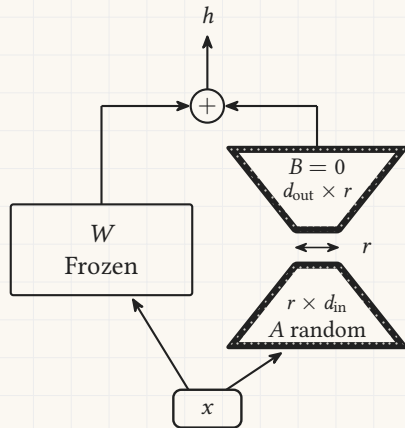
$$h = Wx$$

Both paths run, and their outputs are added



$$h = Wx + (\alpha/r) BAx$$

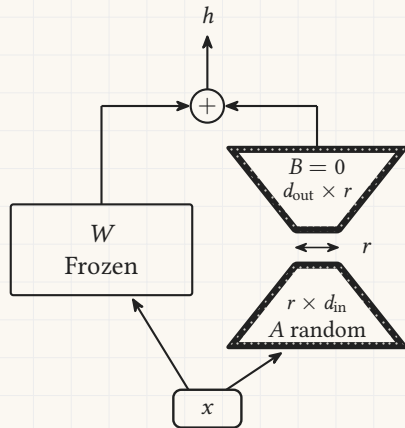
Both paths run, and their outputs are added



$$h = Wx + (\alpha/r) BAx$$

- Both paths read the same x , and their outputs are added

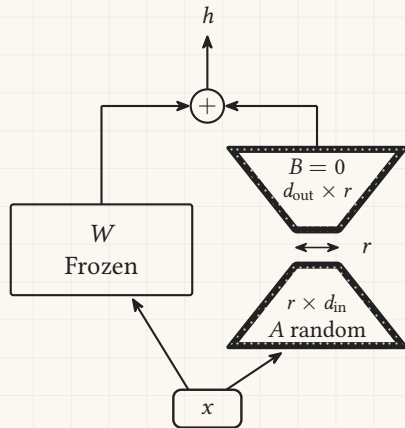
Both paths run, and their outputs are added



$$h = \underbrace{Wx}_{\text{frozen}} + \underbrace{(\alpha/r) BAx}_{\text{trained}}$$

- Both paths read the same x , and their outputs are added

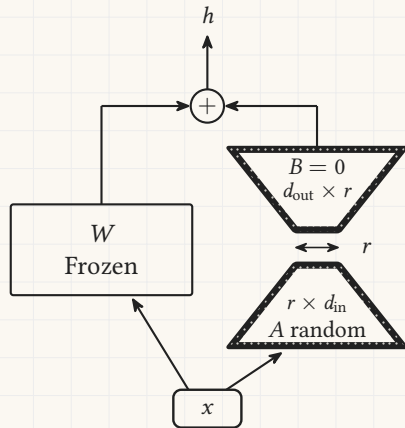
Both paths run, and their outputs are added



$$h = \underbrace{Wx}_{\text{frozen}} + \underbrace{(\alpha/r) BAx}_{\text{trained}}$$

- Both paths read the same x , and their outputs are added
- With Adam, α acts like a learning rate; the $1/r$ lets r change without retuning

Both paths run, and their outputs are added



$$h = \underbrace{Wx}_{\text{frozen}} + \underbrace{(\alpha/r) BAx}_{\text{trained}}$$

- Both paths read the same x , and their outputs are added
- With Adam, α acts like a learning rate; the $1/r$ lets r change without retuning
- The paper's own tables use $\alpha/r = 1, 2$ and 8 ; the PEFT default is $r = 8, \alpha = 8$

Doing the addition

W , frozen

1	2	0
1	0	0
0	2	1

Doing the addition

W , frozen

1	2	0
1	0	0
0	2	1

After training

	1	2	0	A
B	0			
	2			
	1			

Doing the addition

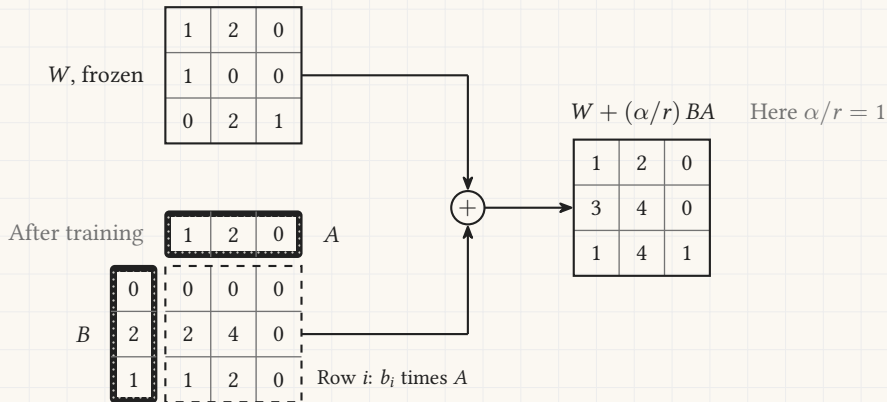
W , frozen

1	2	0
1	0	0
0	2	1

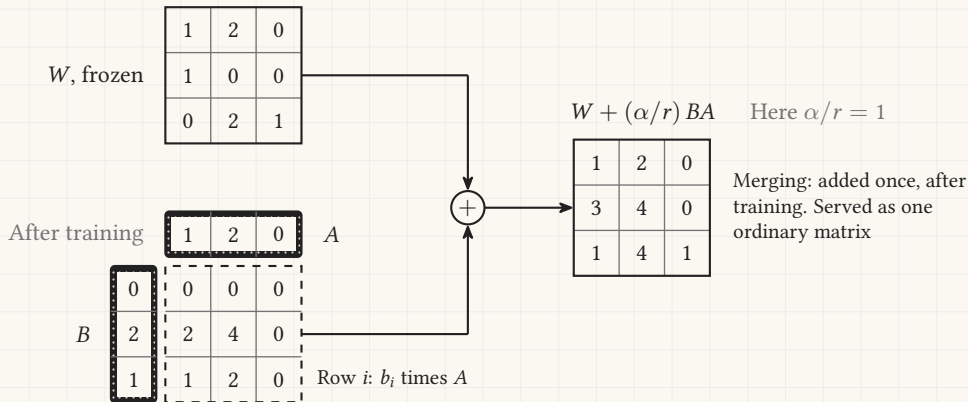
After training

	1	2	0	A
B	0	0	0	0
	2	2	4	0
	1	1	2	0
	Row i : b_i times A			

Doing the addition



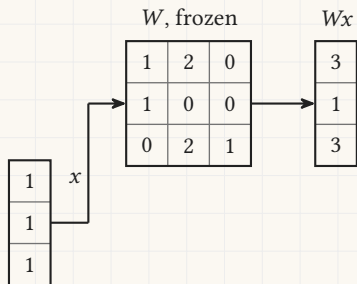
Doing the addition



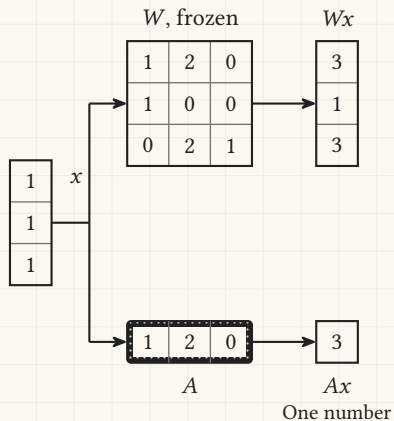
Two paths while training, one matrix after

$$\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} x$$

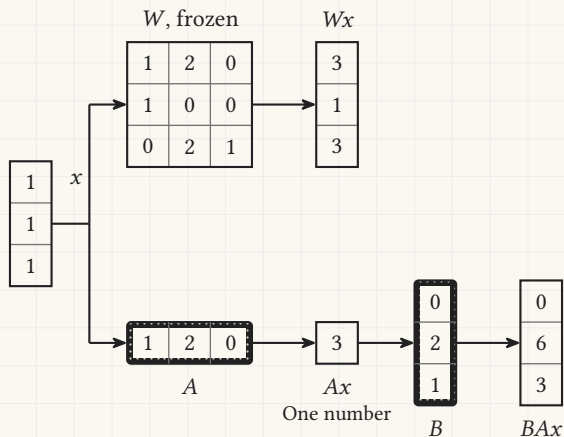
Two paths while training, one matrix after



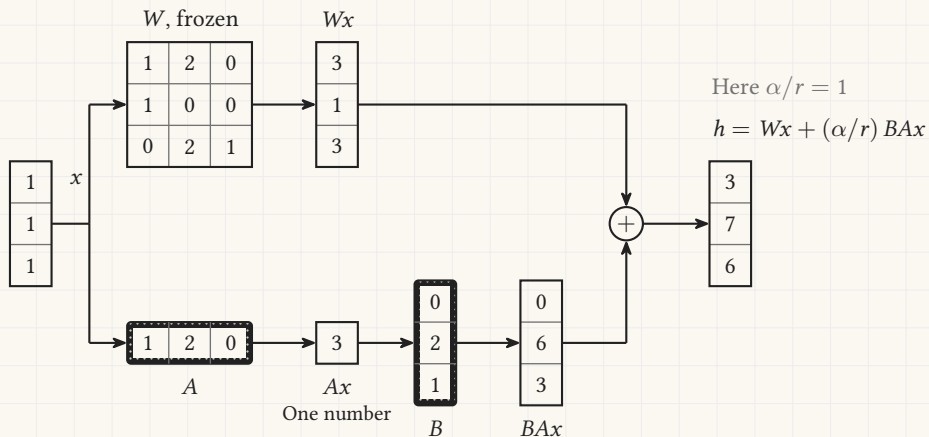
Two paths while training, one matrix after



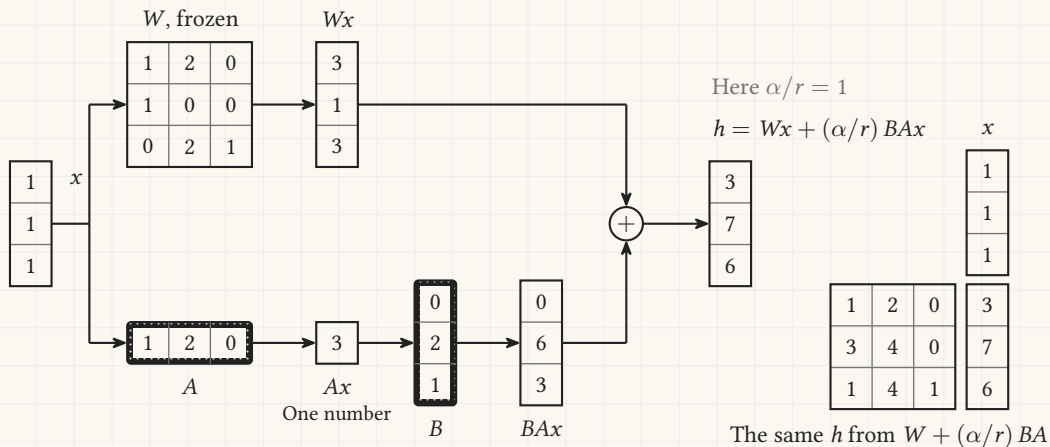
Two paths while training, one matrix after



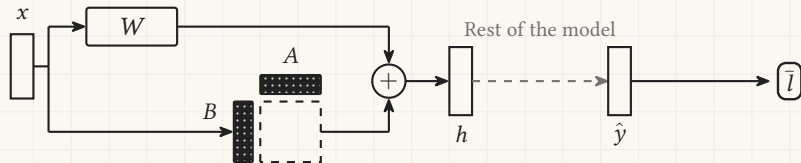
Two paths while training, one matrix after



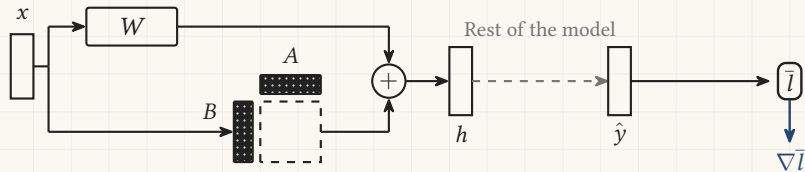
Two paths while training, one matrix after



Gradient descent moves only A and B



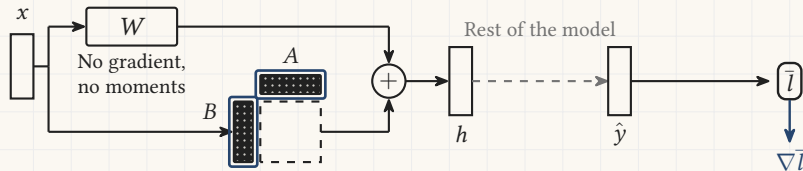
Gradient descent moves only A and B



Full fine-tuning, W trained: 9 entries

$\partial \bar{l} / \partial w_{11}$	$\partial \bar{l} / \partial w_{12}$	$\partial \bar{l} / \partial w_{13}$	$\partial \bar{l} / \partial w_{21}$	$\partial \bar{l} / \partial w_{22}$	$\partial \bar{l} / \partial w_{23}$	$\partial \bar{l} / \partial w_{31}$	$\partial \bar{l} / \partial w_{32}$	$\partial \bar{l} / \partial w_{33}$
--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------

Gradient descent moves only A and B



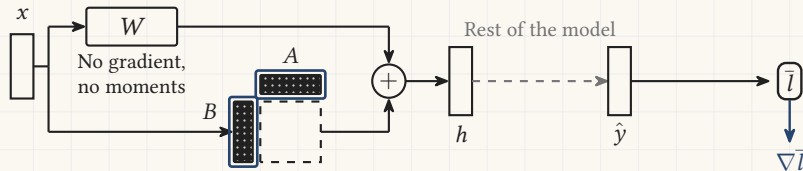
Full fine-tuning, W trained: 9 entries

$\partial \bar{l} / \partial w_{11}$	$\partial \bar{l} / \partial w_{12}$	$\partial \bar{l} / \partial w_{13}$	$\partial \bar{l} / \partial w_{21}$	$\partial \bar{l} / \partial w_{22}$	$\partial \bar{l} / \partial w_{23}$	$\partial \bar{l} / \partial w_{31}$	$\partial \bar{l} / \partial w_{32}$	$\partial \bar{l} / \partial w_{33}$
--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------

LoRA, W frozen: 6 entries

$\partial \bar{l} / \partial b_1$	$\partial \bar{l} / \partial b_2$	$\partial \bar{l} / \partial b_3$	$\partial \bar{l} / \partial a_1$	$\partial \bar{l} / \partial a_2$	$\partial \bar{l} / \partial a_3$
-----------------------------------	-----------------------------------	-----------------------------------	-----------------------------------	-----------------------------------	-----------------------------------

Gradient descent moves only A and B



Full fine-tuning, W trained: 9 entries

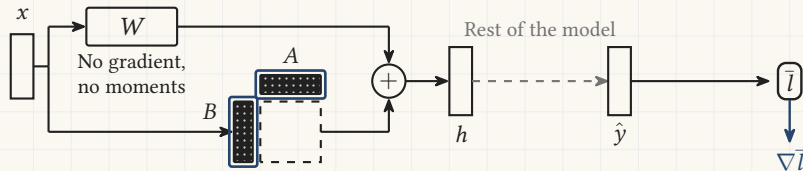
$\partial \bar{l} / \partial w_{11}$	$\partial \bar{l} / \partial w_{12}$	$\partial \bar{l} / \partial w_{13}$	$\partial \bar{l} / \partial w_{21}$	$\partial \bar{l} / \partial w_{22}$	$\partial \bar{l} / \partial w_{23}$	$\partial \bar{l} / \partial w_{31}$	$\partial \bar{l} / \partial w_{32}$	$\partial \bar{l} / \partial w_{33}$
--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------

LoRA, W frozen: 6 entries

$\partial \bar{l} / \partial b_1$	$\partial \bar{l} / \partial b_2$	$\partial \bar{l} / \partial b_3$	$\partial \bar{l} / \partial a_1$	$\partial \bar{l} / \partial a_2$	$\partial \bar{l} / \partial a_3$
-----------------------------------	-----------------------------------	-----------------------------------	-----------------------------------	-----------------------------------	-----------------------------------

$$b \leftarrow b - \eta \frac{\partial \bar{l}}{\partial b} \quad a \leftarrow a - \eta \frac{\partial \bar{l}}{\partial a}$$

Gradient descent moves only A and B



Full fine-tuning, W trained: 9 entries

$\partial \bar{l} / \partial w_{11}$	$\partial \bar{l} / \partial w_{12}$	$\partial \bar{l} / \partial w_{13}$	$\partial \bar{l} / \partial w_{21}$	$\partial \bar{l} / \partial w_{22}$	$\partial \bar{l} / \partial w_{23}$	$\partial \bar{l} / \partial w_{31}$	$\partial \bar{l} / \partial w_{32}$	$\partial \bar{l} / \partial w_{33}$
--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------

At 4096×4096 :
16,777,216 entries

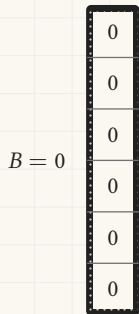
LoRA, W frozen: 6 entries

$\partial \bar{l} / \partial b_1$	$\partial \bar{l} / \partial b_2$	$\partial \bar{l} / \partial b_3$	$\partial \bar{l} / \partial a_1$	$\partial \bar{l} / \partial a_2$	$\partial \bar{l} / \partial a_3$
-----------------------------------	-----------------------------------	-----------------------------------	-----------------------------------	-----------------------------------	-----------------------------------

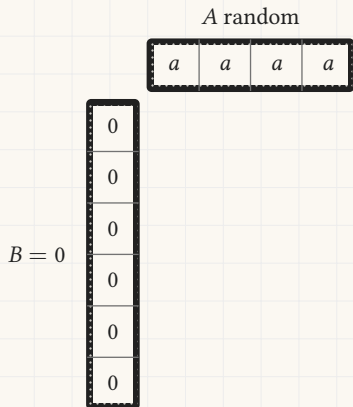
At $r = 8$: 65,536 entries, a factor of 256

$$b \leftarrow b - \eta \frac{\partial \bar{l}}{\partial b} \quad a \leftarrow a - \eta \frac{\partial \bar{l}}{\partial a}$$

B starts at zero, A starts random



B starts at zero, A starts random



B starts at zero, A starts random

A random

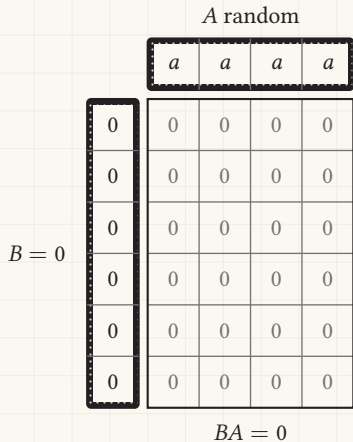
	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

$B = 0$

$BA = 0$

- The adapter does nothing at all on step one

B starts at zero, A starts random

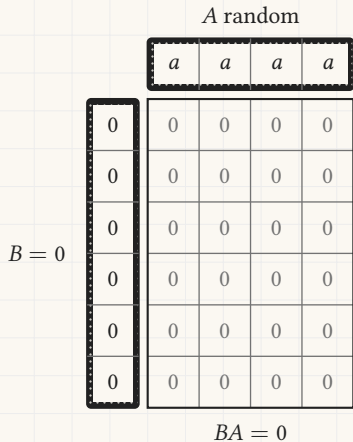


- The adapter does nothing at all on step one

$$W + (\alpha/r) BA = W$$

The first forward pass is exactly the pretrained model

B starts at zero, A starts random



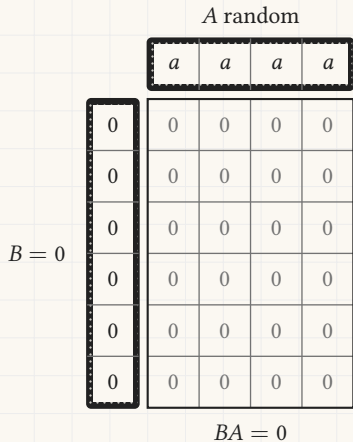
- The adapter does nothing at all on step one

$$W + (\alpha/r) BA = W$$

The first forward pass is exactly the pretrained model

- So training starts exactly where pretraining stopped, even though A is random

B starts at zero, A starts random



- The adapter does nothing at all on step one

$$W + (\alpha/r) BA = W$$

The first forward pass is exactly the pretrained model

- So training starts exactly where pretraining stopped, even though *A* is random
- If both started at zero, both gradients would be zero and nothing would ever move

Rank three costs more than the matrix

Numbers: $r(6 + 4)$

The matrix itself: 24

$r = 1$

10

⋮

Rank three costs more than the matrix

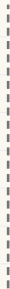
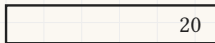
Numbers: $r(6 + 4)$

The matrix itself: 24

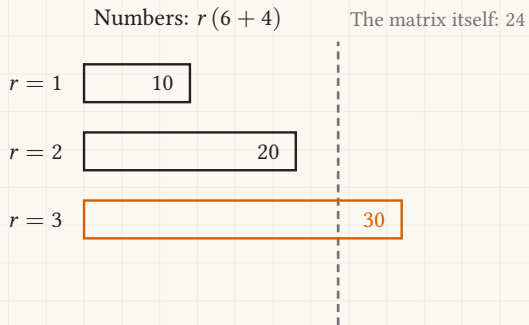
$r = 1$



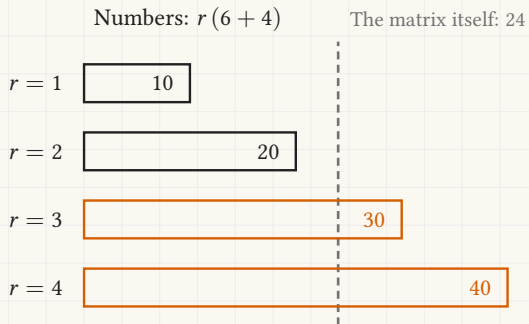
$r = 2$



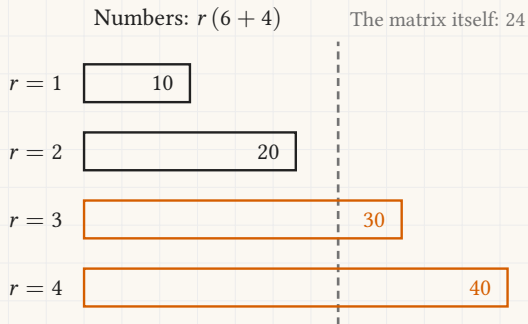
Rank three costs more than the matrix



Rank three costs more than the matrix

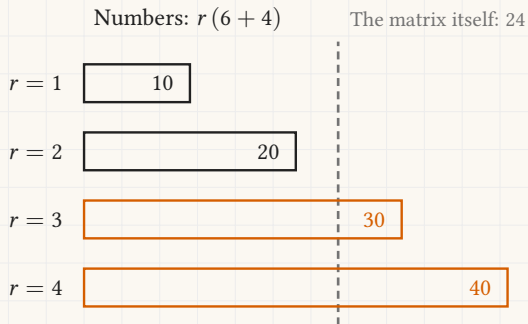


Rank three costs more than the matrix



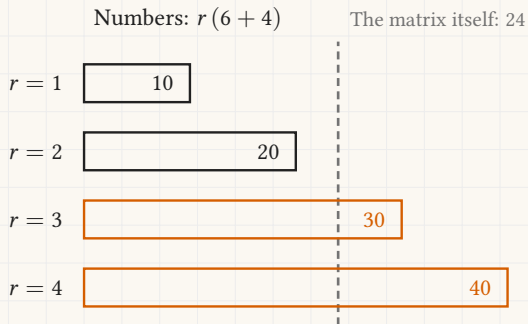
- Break even on this matrix:
 $r = d_{\text{out}} d_{\text{in}} / (d_{\text{out}} + d_{\text{in}}) = 2.4$

Rank three costs more than the matrix



- Break even on this matrix:
 $r = d_{\text{out}}d_{\text{in}}/(d_{\text{out}} + d_{\text{in}}) = 2.4$
- On a square $d \times d$ matrix that is
 $r = d/2$, and the saving is $d/(2r)$

Rank three costs more than the matrix



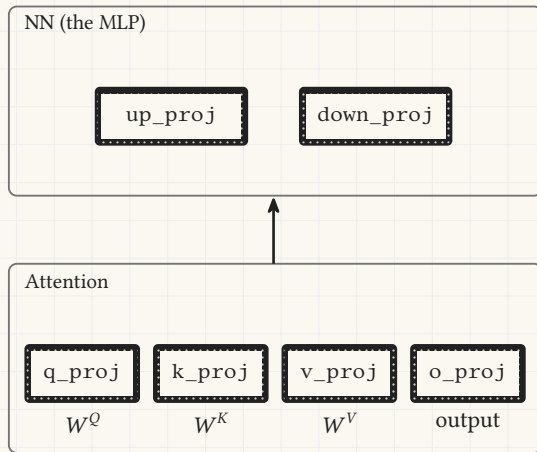
- Break even on this matrix:
 $r = d_{\text{out}}d_{\text{in}}/(d_{\text{out}} + d_{\text{in}}) = 2.4$
- On a square $d \times d$ matrix that is
 $r = d/2$, and the saving is $d/(2r)$
- Llama-2-7B's 4096×4096 query projection at $r = 8$: 256 times fewer

4

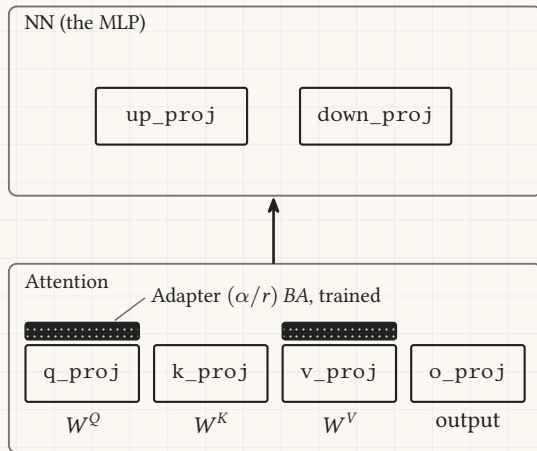
Section 4

The same picture at real scale

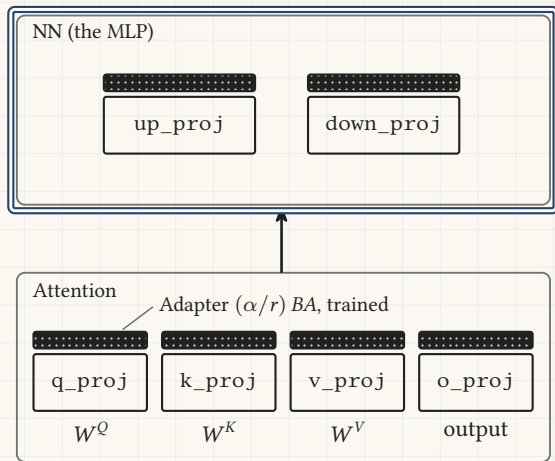
Where the adapters go



Where the adapters go



Where the adapters go



Rank 16 on q and v: 0.08 percent

Apertus-8B's config: 32 layers, $d_{\text{model}} = 4096$, 8 key and value heads

8.05×10^9 parameters

Rank 16 on q and v: 0.08 percent

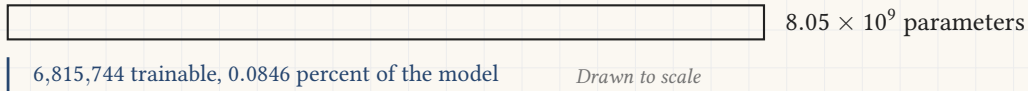
Apertus-8B's config: 32 layers, $d_{\text{model}} = 4096$, 8 key and value heads

8.05×10^9 parameters

| 6,815,744 trainable, 0.0846 percent of the model

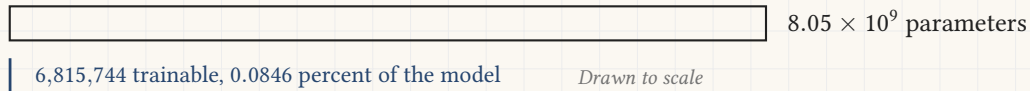
Rank 16 on q and v: 0.08 percent

Apertus-8B's config: 32 layers, $d_{\text{model}} = 4096$, 8 key and value heads



Rank 16 on q and v: 0.08 percent

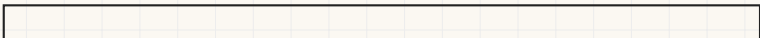
Apertus-8B's config: 32 layers, $d_{\text{model}} = 4096$, 8 key and value heads



Target modules	Rank	Trainable	Percent
q_proj, v_proj (PEFT's default for Llama and Qwen2)	16	6,815,744	0.0846

Rank 16 on q and v: 0.08 percent

Apertus-8B's config: 32 layers, $d_{\text{model}} = 4096$, 8 key and value heads

 8.05×10^9 parameters

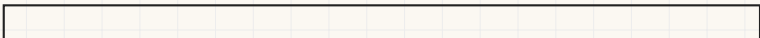
6,815,744 trainable, 0.0846 percent of the model

Drawn to scale

Target modules	Rank	Trainable	Percent
q_proj, v_proj (PEFT's default for Llama and Qwen2)	16	6,815,744	0.0846
All six linear layers	16	39,845,888	0.4948

Rank 16 on q and v: 0.08 percent

Apertus-8B's config: 32 layers, $d_{\text{model}} = 4096$, 8 key and value heads

 8.05×10^9 parameters

6,815,744 trainable, 0.0846 percent of the model

Drawn to scale

Target modules	Rank	Trainable	Percent
q_proj, v_proj (PEFT's default for Llama and Qwen2)	16	6,815,744	0.0846
All six linear layers	16	39,845,888	0.4948
All six linear layers	64	159,383,552	1.9791
All six linear layers	256	637,534,208	7.9164